更好的图像绘制（ImageBitmap）

在这之前我们曾介绍过图像的绘制：图像绘制 。

https://codepen.io/huanjinliu/embed/XWyELjo

之前我们都是直接使用 `new Image()` 作为图像源进行绘制，这里介绍一种更好的方式：`ImageBitmap`！

`ImageBitmap` 代码示例

https://codepen.io/huanjinliu/embed/QWJmXpQ

从示例中我们可以看到相比上方直接使用 `image` 绘制，只是多了下面的语句用于创建 `ImageBitmap` 对象：

```
// 创建一个ImageBitmap对象，返回一个promise
createImageBitmap(image).then(bitmap => {
  ctx.drawImage(bitmap, 0, 0);
});
```

那它具体好在哪？

`ImageBitmap` vs `Image`

在 `Image` 存在的情况下还存在 `ImageBitmap` ，必然有更胜一筹的地方。

① 更高的性能

`ImageBitmap` 对象在创建时就已经解码了图像数据，因此在绘制到Canvas时，不需要再进行解码，可以提高绘制性能。

> 这里提一下，普通的Image对象在创建时并不会解码图像数据，也不存储解码后的像素数据。图像数据的解码通常会在Image对象被绘制到Canvas上时进行，这意味着每次在Canvas上绘制Image对象时，不管是否已绘制过，都需要重新进行解码，相比而言更可能影响到绘制的性能。
>
> 这里"解码"的意思指的是：图像文件从网络或磁盘加载时为了减少文件大小会进行编码，而解码也就是转换回原始的像素数据的过程。

② 可结合Web Worker使用

`Image` 对象不能传入Web Worker，但 `ImageBitmap` 对象却可以。

除此 `ImageBitmap` 对象还有另一个极大的优势，那就是它传入worker线程的方式可以通过转移实现，而不需要通过拷贝实现。

一般worker线程和主线程之间传输信息/数据是要保证数据是可序列化的，简单来讲就是支持 `JSON.parse(JSON.stringify(data))` 的数据。换句话说worker线程和主线程直接的数据传输是通过数据拷贝，而不是共享来完成的。所以我们之前提到的通过 `getImageData` 获取图像像素数据后再通过 `postMessage(imageData)` 传入worker线程本身这个操作就比较低效。

但是！ `postMessage` 方法还有第二个参数 `transferList` ：

```
createImageBitmap(image).then(bitmap => {
  postMessage(bitmap, [bitmap]);
});
```

我们可以通过这个参数设置允许转移所有权的对象，比如 `ImageBitmap` 对象，这些对象的所有权将被转移到接收消息的线程，而在发送消息的线程中，这些对象将不再可用，但是因为不用拷贝多一份数据，只是转移了所有权，所以可以大大提高主线程和worker线程间数据传输的效率。

思考：为什么普通 `Image` 对象不能传入Web Worker?

解答：

我们之前说过：“Web Worker是不能直接访问主线程的全局环境或 DOM”。

Web Worker是运行在与主线程隔离的环境中，无法访问DOM，但是 `Image` 对象却包含一些与DOM相关的属性和方法，所以是不被支持的。

但是 `ImageBitmap` 对象在创建时就已经解码了图像数据，且不包含任何与DOM相关的属性或方法。因此，它可以安全地在Web Worker中使用。

③ 可以手动释放图像占用内存

`ImageBitmap` 有一个 `close` 方法可以手动释放对象占用内存。

```
createImageBitmap(image).then(bitmap => {
  bitmap.close();
});
```

④ 更多的图像配置：

`createImageBitmap` 工厂方法有第二个参数，可以配置控制图像的解码和裁剪，比如：

```
const options = {
```

```
    resizeWidth: 100,                // 调整宽度
    resizeHeight: 100,               // 调整高度
    resizeQuality: 'high'            // 调整图像质量，默认是low
};
createImageBitmap(image, options).then(function(bitmap) {
    const canvas = document.getElementById('myCanvas');
    const ctx = canvas.getContext('2d');
    ctx.drawImage(bitmap, 0, 0);
});
```

当然不止我说的这些，更多可查阅：MDN。